Developer Report – Engineering Analysis Suite

Multi-page Streamlit Application (Modules A–D)

1. Interesting Engineering Insight Revealed by the App

When exploring the Pipe Flow Analyser, one quickly observes that pressure drop scales approximately with the square of flow rate in the turbulent regime ($\Delta P \propto V^2 \propto Q^2$). However, the interactive plot also shows that the friction factor itself slowly decreases with increasing Reynolds number. Consequently the actual exponent is slightly less than 2. This non-linearity becomes practically important when sizing pumps or estimating energy costs for variable-demand pipelines: doubling the flow rate more than doubles the required pumping power, but not quite by a factor of eight (because f also changes). The app makes this classic fluid-mechanics behaviour immediately visible and quantifiable.

2. Technical Challenge Overcome

The main technical challenge was obtaining a robust, non-iterative friction-factor calculation that remains accurate across laminar, transitional and fully turbulent regimes while staying transparent for educational use. The Colebrook–White equation is implicit; many published explicit approximations exist. After verifying several options against hand calculations, the Haaland approximation was selected for its excellent accuracy and simple algebraic form. A clean laminar branch ($f = 64/Re$) was added, and careful input validation plus informative error messages were implemented so that physically impossible geometries or negative flow rates never crash the application. Separating the engineering logic into a well-documented OOP module (engineering.py) further improved maintainability and testability.

3. What I Would Add With More Time

With additional development time I would extend Module A to support minor-loss coefficients (valves, bends, entrances) and a simple pump-curve overlay so users can perform a complete system-curve analysis. For Module B I would add the two-layer composite-wall conduction problem and a Biot-number check that automatically warns when the lumped-capacitance assumption underlying Newton's law of cooling becomes invalid. Finally, Module C could benefit from a simple machine-learning permeability predictor trained on the uploaded data, giving students a gentle introduction to data-driven petrophysics.

Repository & Deployment

GitHub repository: (push the local pipe_flow_app folder to your GitHub account)
Live Streamlit app: (deploy via share.streamlit.io pointing at streamlit_app.py)